

SLOT 15

Chapter 12: Versioning, Paging, Sorting and Testing

Part A – API Versioning • Paging • Sorting • Page/Slice + Lab 03 Kick-off

CLO3 – Describe RESTful APIs using Spring Boot built-in features and frameworks

Nền tảng: JDK 21 • Spring Boot 3 • Maven • Spring Web • Spring Data JPA • H2 • Postman

Phạm vi: Chapter 12, slide 1–19

Mental model xuyên suốt: API đã chạy và được document → API thay đổi cần version → collection lớn cần paging → client cần sorting → chọn Page hay Slice theo nhu cầu metadata/performance → chuẩn bị Lab 03.

Engineering rule: Không coi pagination chỉ là “thêm page/size”. Phải kiểm soát page index, page size tối đa, sort fields, response contract và tác động đến database query.

CÁCH SỬ DỤNG GIÁO TRÌNH

Tài liệu tiếp tục template của Slot 12–14 theo mạch Prepare → Understand → Observe → Build → Break → Verify → Research → Transfer. Buổi học tập trung nâng cấp API đã có; testing tự động chỉ preview và được dành cho Slot 16.

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Ôn REST, CRUD, OpenAPI; kiểm tra JDK/Maven/H2/Postman.	3-2-1 note + environment check
Understand	Giải thích breaking change, paging, sorting, Page/Slice.	Concept map
Observe	Gọi endpoint lớn, đọc query params và page metadata.	Postman trace
Build	Version endpoints; implement paging/sorting với Spring Data.	Source + responses
Break	Gửi page âm, size quá lớn, sort field sai, version sai.	Bug log
Verify	Đối chiếu response, SQL/query behavior, metadata.	Verification matrix
Research	AI prompts có kiểm chứng.	AI evidence log
Transfer	Áp dụng cho FUNews + Lab 03.	API upgrade plan

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1 Ôn lại Slot 14 và nền tảng trước đó

- OpenAPI/Swagger và springdoc-openapi: endpoint, parameter, schema, response phải được document rõ.
- Slot 13: Spring Boot annotations, Controller → Service → Repository, CRUD, request/response handling.
- REST: resource, URI, HTTP method, status code, representation.
- Java: interface, generics, List, Optional; Maven dependency cơ bản.
- Spring Data JPA ở mức bridge: repository abstraction, entity đơn giản. Không yêu cầu quan hệ JPA phức tạp trong Slot 15.

0.2 Công cụ cần chuẩn bị

Công cụ	Vai trò	Kiểm tra nhanh
JDK 21	Compile/run Spring Boot	java -version
Maven	Build/dependency	mvn -v
IntelliJ IDEA	IDE	Maven sync không lỗi
Spring Initializr	Tạo project	Spring Web + Data JPA + H2
Postman	Manual API verification	GET localhost hoạt động
H2	Database demo nhẹ	App start, schema tạo được
Browser	Swagger UI nếu giữ Slot 14	/swagger-ui/index.html

0.3 Pre-class 3-2-1 Learning Note

- 3 lý do không nên trả toàn bộ 10.000 records trong một response.
- 2 tình huống API thay đổi có thể làm client cũ hỏng.
- 1 câu hỏi: Khi nào UI cần totalPages, khi nào chỉ cần “Load more”?

0.4 Câu hỏi tự kiểm tra

1. Page index mặc định trong Spring Data web thường bắt đầu từ 0 hay 1?
2. Query parameter `sort=publishDate,desc` thể hiện điều gì?
3. Nếu response v2 thêm field mới thì luôn là breaking change không?
4. Nếu UI chỉ có nút “Load more”, có nhất thiết phải COUNT toàn bộ bảng không?
5. Version nên nằm ở path, query, header hay media type? Vì sao?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Tóm tắt kiến thức Slot 14

Chủ đề	Đã làm được	Giới hạn còn lại	Slot 15 giải quyết
API Documentation	Mô tả endpoint/contract	API thay đổi theo thời gian	Versioning
OpenAPI/Swagger	Khám phá/test API	List lớn vẫn nặng	Paging
Query parameters	Document/filter cơ bản	Cần page/size/sort chuẩn	Pageable/Sort
3-layer CRUD	CRUD API chạy được	Collection chưa tối ưu	Spring Data paging/sorting

2. Định vị Slot 15

Slot	Chủ đề	Mối liên hệ
12	REST fundamentals	Thiết kế resource/method/status.
13	Annotations + 3-layer CRUD	Triển khai API.
14	OpenAPI/Swagger	Document contract.
15	Versioning + paging + sorting	Nâng cấp API list và evolution.
16	Testing + MockMvc + Lab 03	Kiểm thử behavior của API đã nâng cấp.
17	JPA/Entity/Repository/Service	Đi sâu persistence/data model.
18	REST Controller + Lab 04	Hoàn thiện integration backend.

3. Bối cảnh bài học – FUNewsManagementSystem

Giả sử `GET /api/news` ban đầu chỉ có vài chục bản ghi. Sau một năm hệ thống có 10.000+ tin, frontend cần trang 1/2/3, sort theo ngày mới nhất và vẫn phải hỗ trợ mobile client cũ khi response mới được mở rộng. Slot 15 giải quyết ba bài toán production cơ bản: API evolution, pagination và sorting.

4. Mục tiêu sau buổi học

1. Giải thích backward compatibility và khi nào cần API versioning.
2. Phân biệt path/URI, query parameter, custom header và media type/Accept versioning.
3. Thiết kế paging contract gồm page, size, sort và giới hạn size.
4. Sử dụng Pageable, PageRequest và Sort.
5. Hiểu repository support cho paging/sorting trong Spring Data hiện đại.
6. Phân biệt Page và Slice về metadata, count query và use case.

7. Triển khai demo News API có versioning + paging + sorting.
8. Bắt đầu Lab 03 với checklist/evidence rõ ràng, chưa đi sâu automated testing.

5. Khái niệm trung tâm

Khái niệm	Một câu cần nhớ
API Versioning	Cho phép API tiến hóa mà không phá client đang dùng contract cũ.
Backward compatibility	Client cũ vẫn hoạt động đúng sau khi server thay đổi.
Pageable	Mô tả page number, size và Sort cho một request dữ liệu.
Sort	Mô tả thứ tự sắp xếp theo một hoặc nhiều property.
Page	Slice + tổng số phần tử/trang; thường cần count query.
Slice	Biết content và còn trang sau hay không; không cần total count.
PageRequest	Implementation tạo Pageable theo page/size/sort.
Page size guard	Giới hạn size để bảo vệ database/server khỏi request quá lớn.

6. Flow tổng quát

Flow: CRUD API → version contract → receive page/size/sort → Pageable → Repository → Page/Slice → response metadata → frontend pagination/infinite scroll → Slot 16 testing.

7. Tư duy kiến trúc quan trọng

- Versioning chỉ cần khi có thay đổi contract đáng kể; không tạo v2 chỉ vì refactor nội bộ.
- Pagination contract là phần public API, vì vậy phải document page base, max size, sort syntax và response metadata.
- Không cho client sort tùy ý trên mọi property; whitelist field được phép để tránh lỗi hoặc query ngoài dự kiến.
- Page phù hợp UI cần total pages; Slice phù hợp “next/load more” và dataset lớn.
- Repository là chi tiết data access; controller không tự xây SQL/paging logic.

8. Tài liệu tham khảo

- Slide Chapter 12, trang 1–19.
- Spring Data Commons/JPA reference về Pageable, Sort, Page/Slice và web support.
- Spring Framework reference về API versioning (ghi chú cập nhật; course vẫn dùng Spring Boot 3).
- Bootify – Pagination in Spring Boot REST API.
- Spring REST: Building Java Microservices and Cloud Applications – Chapter 07.
- Giáo trình Slot 13–14 SBA301.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. Vì sao cần API Versioning?

- Business requirements thay đổi: field mới, validation mới, representation mới.
- Client được phát hành độc lập: web, mobile, partner systems có nhịp nâng cấp khác nhau.
- Breaking change có thể làm client cũ parse sai, gửi payload sai hoặc mất chức năng.
- Versioning tạo “đường chuyển tiếp” thay vì ép mọi client nâng cấp cùng lúc.

Không phải thay đổi nào cũng cần version mới: Thêm một optional response field thường có thể backward-compatible; đổi tên/xóa field, đổi semantics hoặc đổi type thường có nguy cơ breaking.

2. Chuẩn hóa 4 chiến lược versioning

Chiến lược	Ví dụ	Ưu điểm	Hạn chế
Path / URI	/api/v1/news	Rõ ràng, dễ cache/log	URL thay đổi theo version
Query parameter	/api/news?version=1	Giữ base path	Dễ bị xem như filter hơn contract
Custom header	X-API-Version: 1	URI sạch	Khó nhìn bằng browser/link
Media type / Accept	Accept: application/vnd.funews.v1+json	Gắn version vào representation	Phức tạp hơn khi debug/client config

Technical normalization: Trong prompt gốc hai ví dụ header dễ bị đảo tên. Tài liệu này dùng: custom header = `X-API-Version` ; Accept/media type = vendor media type trong `Accept`.

3. Code minh họa versioning trong Spring Boot 3

3.1 Path/URI versioning

```
@RestController
@RequestMapping("/api")
public class NewsVersionController {

    @GetMapping("/v1/news")
    public List<NewsV1> getV1() {
        return List.of(new NewsV1(1L, "Hello REST"));
    }

    @GetMapping("/v2/news")
    public List<NewsV2> getV2() {
        return List.of(new NewsV2(1L, "Hello REST", true));
    }
}

record NewsV1(Long id, String title) {}
record NewsV2(Long id, String title, boolean active) {}
```

3.2 Query parameter versioning

```
@GetMapping(value = "/news", params = "version=1")
public List<NewsV1> newsV1ByParam() { ... }

@GetMapping(value = "/news", params = "version=2")
public List<NewsV2> newsV2ByParam() { ... }
```

3.3 Custom header versioning

```
@GetMapping(value = "/news", headers = "X-API-Version=1")
public List<NewsV1> newsV1ByHeader() { ... }

@GetMapping(value = "/news", headers = "X-API-Version=2")
public List<NewsV2> newsV2ByHeader() { ... }
```

3.4 Accept/media-type versioning

```
@GetMapping(value = "/news", produces = "application/vnd.funews.v1+json")
public List<NewsV1> newsV1ByMediaType() { ... }

@GetMapping(value = "/news", produces = "application/vnd.funews.v2+json")
public List<NewsV2> newsV2ByMediaType() { ... }
```

2026 framework note: Spring Framework 7 / Spring Boot 4 bổ sung first-class API versioning. Khóa học dùng Spring Boot 3 nên demo không phụ thuộc API mới đó; sinh viên học chiến lược versioning trước, framework-native support là nội dung research/update.

4. Vì sao cần paging?

Vấn đề khi trả tất cả	Hệ quả	Paging cải thiện
10.000–1.000.000 rows	Response lớn, memory/CPU cao	Chỉ lấy chunk cần dùng
Network chậm/mobile	Tải lâu, tốn bandwidth	Payload nhỏ hơn
UI table	Render quá nhiều rows	10–50 rows/page
Database query	Scan/transfer nhiều	LIMIT/OFFSET hoặc chiến lược tương đương

5. Paging contract cơ bản

HTTP request mẫu

```
GET /api/news?page=0&size=10&sort=publishDate,desc
Accept: application/json
```

Parameter	Ý nghĩa	Quy ước khuyến nghị
page	Trang cần lấy	0-based; default 0
size	Số phần tử/trang	default 20; max ví dụ 100
sort	property,direction	whitelist property; asc/desc
sort lặp	Nhiều tiêu chí	?sort=active,desc&sort=publishDate,desc

Default Spring Data web support: `page` được hiểu là 0-indexed; `size` và `sort` được resolve thành Pageable/Sort. Tuy nhiên ứng dụng vẫn phải validate business constraints như max size và sort field.

6. Spring Data repository – technical correction

Trong Spring Data hiện đại, `PagingAndSortingRepository` là một repository fragment chuyên cung cấp `findAll(Pageable)` và `findAll(Sort)`; nó không kế thừa `JpaRepository`. Trong ứng dụng Spring Data JPA, cách phổ biến nhất là `extends JpaRepository<Entity, Id>` vì `JpaRepository` đã tích hợp các khả năng cần thiết cho CRUD và paging/sorting.

```
public interface NewsRepository extends JpaRepository<News, Long> {
    Slice<News> findByActive(boolean active, Pageable pageable);
}
```

Teaching note: Vẫn cần biết tên `PagingAndSortingRepository` vì slide đề cập và nó giải thích abstraction. Nhưng demo dùng `JpaRepository` để tránh tạo repository thiếu CRUD do hiểu nhầm inheritance.

7. Pageable, PageRequest và Sort

```
Sort sort = Sort.by(Sort.Direction.DISC, "publishDate");
Pageable pageable = PageRequest.of(0, 10, sort);
Page<News> result = newsRepository.findAll(pageable);
```

Object	Trả lời câu hỏi gì?
Pageable	Trang nào? kích thước bao nhiêu? sort thế nào?
PageRequest	Cách tạo Pageable cụ thể.
Sort	Sort theo property nào, hướng nào?
Page<T>	Content + metadata tổng thể.
Slice<T>	Content + hasNext/hasPrevious, không cần total count.

8. Page vs Slice

Feature	Page	Slice
Content	Có	Có
number/size	Có	Có
hasNext/hasPrevious	Có	Có
totalElements	Có	Không
totalPages	Có	Không
Count query	Thường cần	Không cần total count
Use case	Numbered pagination	Load more / infinite scroll
Trade-off	Metadata đầy đủ, có thể tốn thêm query	Nhẹ hơn nhưng không biết tổng số

8.1 Page query

```
public Page<News> getNews(Pageable pageable) {
    return newsRepository.findAll(pageable);
}
```

8.2 Slice query

```
public Slice<News> getActiveNewsSlice(Pageable pageable) {
    return newsRepository.findByActive(true, pageable);
}
```

9. Page response contract – classroom vs production

Trong lab cơ bản, trả trực tiếp `Page<News>` giúp sinh viên quan sát metadata. Tuy nhiên Spring Data hiện khuyến nghị không phụ thuộc JSON của implementation `PageImpl` như một public contract lâu dài. Ứng dụng thật nên dùng response DTO/PagedModel để contract ổn định.

```
public record PageResponse<T>(
    List<T> content,
    int page,
    int size,
    long totalElements,
    int totalPages,
    boolean last
) {
    static <T> PageResponse<T> from(Page<T> p) {
        return new PageResponse<>(
            p.getContent(), p.getNumber(), p.getSize(),
            p.getTotalElements(), p.getTotalPages(), p.isLast()
        );
    }
}
```

```

        p.getTotalElements(), p.getTotalPages(), p.isLast());
    }
}

```

10. Sorting – validate thay vì nhận mọi field

```

private static final Set<String> ALLOWED_SORTS =
    Set.of("id", "title", "publishDate", "active");

private String safeSort(String requested) {
    if (!ALLOWED_SORTS.contains(requested)) {
        throw new IllegalArgumentException("Unsupported sort field");
    }
    return requested;
}

```

Security/performance habit: Spring Data nhận property name từ request nhưng không tự xác nhận đó là business-approved field. Hãy whitelist sort fields và giới hạn size.

11. Những hiểu sai phổ biến

Hiểu sai	Điều chỉnh
Page number bắt đầu từ 1	Spring Data web default là 0-based.
Page và Slice chỉ khác tên	Page có totalElements/totalPages; Slice không.
Page luôn tốt hơn	Count query có thể đắt; Slice hợp “load more”.
PagingAndSortingRepository extends JpaRepository	Không; đây là fragment riêng. JPA app thường dùng JpaRepository.
Sort field nào cũng cho client dùng	Cần whitelist/validation.
Versioning bắt buộc từ v1 cho mọi API	Phụ thuộc public contract và evolution strategy.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM THỰC TẾ

1. Mapping use case → kỹ thuật

Sản phẩm/tính huống	Kỹ thuật phù hợp	Lý do
News web table có số trang	Page + sort publishDate desc	Cần totalPages
Mobile feed “Load more”	Slice	Chỉ cần hasNext
Public partner API thay payload	Versioning	Giữ client cũ
E-commerce product catalog	Paging + multi-sort/filter	Dataset lớn
Bank transaction history	Paging + stable sort	Tránh response quá lớn

2. Case study – FUNewsManagementSystem

```

GET /api/v1/news?page=0&size=10&sort=publishDate,desc
GET /api/v2/news?page=0&size=10&sort=publishDate,desc
GET /api/news/slice?active=true&page=0&size=10

```

- v1 có thể trả `id,title`; v2 trả thêm `categoryName,publishDate,active` hoặc đổi contract có kiểm soát.
- Admin web table dùng Page để hiển thị “Page 1 of N”.
- Public/mobile feed có thể dùng Slice để giảm nhu cầu count.

- OpenAPI Slot 14 phải được update để document page/size/sort/version contract.

3. API evolution decision table

Thay đổi	Khả năng breaking	Hành động gợi ý
Thêm optional field response	Thấp	Thường giữ version nếu client tolerant
Đổi tên field	Cao	Version/deprecation plan
Đổi type number → string	Cao	Version
Thêm endpoint mới	Thấp	Không cần version mới
Thay semantics status/field	Cao	Version hoặc migration rõ ràng

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN THỰC HIỆN VÀ TRIỂN KHAI

A. Demo 1 – API Versioning với Spring Boot 3

A1. Mục tiêu

Tạo một controller cho News với v1/v2, sau đó kiểm tra bốn cách resolve version ở mức mapping conditions. Không cần database.

A2. Bước thực hiện

1. Mở project Slot 13/14 hoặc tạo Spring Web project mới.
2. Tạo record `NewsV1`, `NewsV2`.
3. Tạo path-version endpoints `/api/v1/news`, `/api/v2/news`.
4. Thêm query-parameter variant bằng `params`.
5. Thêm custom-header variant bằng `headers`.
6. Thêm media-type variant bằng `produces`.
7. Dùng Postman gửi request và ghi lại mapping nào được chọn.

A3. Break-It mini lab

Lỗi cố ý	Quan sát
Gọi /api/news?version=3	Không có mapping → thường 404
Bỏ X-API-Version	Header mapping không match
Accept sai media type	Có thể 406 Not Acceptable
V1/V2 trả cùng DTO	Version tồn tại nhưng không có ý nghĩa business

B. Demo 2 – Paging + Sorting với Spring Data JPA và H2

B1. Project setup

- Spring Web
- Spring Data JPA
- H2 Database
- Lombok (optional)
- Validation (optional, nếu lớp đã dùng)

application.properties

```
spring.datasource.url=jdbc:h2:mem:slot15db
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
spring.jpa.show-sql=true
```

Scope guard: Entity/JPA ở đây chỉ là cầu nối để minh họa paging trên repository. Mapping quan hệ, transaction, entity design sâu để Slot 17.

B2. Entity News

```
@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class News {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private LocalDate publishDate;
    private boolean active;
}
```

B3. Repository

```
public interface NewsRepository extends JpaRepository<News, Long> {
    Slice<News> findByActive(boolean active, Pageable pageable);
}
```

B4. Seed data

```
@Bean
CommandLineRunner seed(NewsRepository repo) {
    return args -> {
        for (int i = 1; i <= 35; i++) {
            News n = new News();
            n.setTitle("News " + i);
            n.setPublishDate(LocalDate.now().minusDays(i));
            n.setActive(i % 4 != 0);
            repo.save(n);
        }
    };
}
```

B5. Service

```
@Service
@RequiredArgsConstructor
public class NewsService {
    private final NewsRepository repository;

    public Page<News> findPage(Pageable pageable) {
        return repository.findAll(pageable);
    }

    public Slice<News> findActiveSlice(Pageable pageable) {
        return repository.findByActive(true, pageable);
    }
}
```

B6. Controller – nhận Pageable trực tiếp

```
@RestController
@RequestMapping("/api/news")
@RequiredArgsConstructor
public class NewsPagingController {
    private final NewsService service;

    @GetMapping
    public Page<News> getPage(
```

```

    @PageableDefault(size = 10, sort = "publishDate",
        direction = Sort.Direction.DESC) Pageable pageable) {
        return service.findPage(pageable);
    }

    @GetMapping("/slice")
    public Slice<News> getActiveSlice(
        @PageableDefault(size = 10, sort = "publishDate",
            direction = Sort.Direction.DESC) Pageable pageable) {
        return service.findActiveSlice(pageable);
    }
}

```

B7. Postman requests

```

GET http://localhost:8080/api/news?page=0&size=5
GET http://localhost:8080/api/news?page=1&size=5
GET http://localhost:8080/api/news?page=0&size=5&sort=title,asc
GET http://localhost:8080/api/news/slice?page=0&size=5

```

B8. Response trace

Page response cần quan sát	Slice response cần quan sát
content	content
number/page	number/page
size	size
totalElements	—
totalPages	—
first/last	first/last
hasNext qua metadata	hasNext trực tiếp

C. Guardrails – page size và sort whitelist

Automatic `Pageable` rất tiện nhưng production API thường cần guard. Một cách dễ dạy là nhận primitive parameters, validate, rồi tạo `PageRequest` có kiểm soát.

```

@GetMapping("/safe")
public Page<News> getSafePage(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "publishDate") String sortBy,
    @RequestParam(defaultValue = "desc") String direction) {

    int safePage = Math.max(page, 0);
    int safeSize = Math.min(Math.max(size, 1), 100);
    String safeSort = safeSort(sortBy);
    Sort.Direction dir = "asc".equalsIgnoreCase(direction)
        ? Sort.Direction.ASC : Sort.Direction.DESC;

    Pageable pageable = PageRequest.of(
        safePage, safeSize, Sort.by(dir, safeSort));
    return service.findPage(pageable);
}

```

Design question: Nên silently clamp `size=5000` xuống 100 hay trả 400? Cả hai có thể hợp lệ nếu contract rõ; yêu cầu sinh viên chọn và giải thích.

D. Lab 03 – RESTful Web Services with Spring Boot: Kick-off

D1. Mục tiêu Lab 03 trong Slot 15

- Chuẩn bị project và architecture.
- Chọn resource (Employee/News theo đề lab thực tế).
- Hoàn thành versioning + paging + sorting cơ bản.
- Tạo Postman collection/evidence.
- Để phần automated testing/MockMvc sang Slot 16.

D2. Milestones đề xuất

Milestone	Đầu ra	Evidence
L3-V0	Project chạy, dependencies đúng	Terminal + pom.xml
L3-V1	CRUD/base endpoint tồn tại	Postman GET
L3-V2	v1/v2 mapping	2 response samples
L3-V3	Pageable paging	page 0/1 evidence
L3-V4	Sorting	asc/desc evidence
L3-V5	Page vs Slice observation	response comparison
L3-V6	README + API contract	Git/README

D3. Lab 03 readiness gate

1. Project start không lỗi.
2. Không có dependency conflict.
3. Sinh viên giải thích được page=0 và size.
4. Ít nhất 20–30 records để paging có ý nghĩa.
5. Có endpoint Page và một bài quan sát Slice.
6. Có Postman collection.
7. Không triển khai MockMvc trong Slot 15 nếu chưa đến Part B/testing.

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ DEBUGGING

1. Lỗi thường gặp

Triệu chứng	Nguyên nhân thường gặp	Cách kiểm tra
page=1 nhưng tưởng là trang đầu	Nhầm 0-based	Thử page=0 và page=1
Sort không chạy	Sai property name	Đối chiếu entity field
500 khi sort field lạ	Không validate field	Whitelist + handler
Response Page quá phức tạp	Serialize framework type trực tiếp	Dùng DTO/PagedModel
Slice vẫn cần totalPages	Chọn sai abstraction	Đối Page
V1/V2 mapping conflict	Conditions/path chưa đủ khác	Kiểm tra mapping startup log
size=100000 làm chậm	Không max-size guard	Clamp/reject request

2. Debug decision tree

1. Endpoint không match → kiểm tra path/params/headers/produces và response 404/406.
2. Paging sai → log `pageable.getPageNumber()/getPageSize()/getSort()`.
3. Sort lỗi → kiểm tra property Java đúng chính tả và có được whitelist.
4. Page trống → kiểm tra page index có vượt totalPages.
5. Performance chậm → kiểm tra size, query log, count query, sort index/database.
6. Contract JSON không ổn định → dùng DTO/PagedModel thay vì expose implementation.

3. Best practices

- Document rõ 0-based page numbering.
- Có default size và maximum size.
- Dùng stable sort khi pagination quan trọng; có thể thêm id làm secondary sort.
- Whitelist sort properties.
- Không version chỉ để “đẹp”; có migration/deprecation story.
- Không trả toàn bộ entity graph trong page response.
- Page cho numbered UI; Slice cho load-more/infinite scroll.

4. Break-It Lab

Bug injection	Sinh viên phải giải thích
page=-1	Validation/behavior mong muốn?
size=0	Có hợp contract?
size=100000	Resource protection
sort=password,asc	Tại sao phải whitelist?
Accept v3 media type	Server nên trả gì?
GET page vượt range	Empty content hay 404? Chọn contract và giải thích.

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

A. 10 bài tập chủ đề

Bài	Yêu cầu
1. URI versioning	Tạo `/api/v1/employees` và `/api/v2/employees`; v2 có thêm email/department.
2. Query versioning	Dùng `params="version=1/2"`; test missing/unsupported version.
3. Header versioning	Dùng `X-API-Version`; tạo Postman environment.
4. Media type versioning	Dùng vendor media type; quan sát 406 khi Accept không match.
5. Pageable	GET employees?page=0&size=10; giải thích page metadata.
6. Sorting	Sort name asc/desc; test field không tồn tại.
7. Page vs Slice	Tạo 2 endpoint và so sánh metadata/query behavior.
8. Guard page size	Giới hạn 1..100 và chọn clamp hoặc 400.

9. Multi-sort	Sort active desc rồi publishDate desc.
10. Contract design	Viết OpenAPI-style table cho page/size/sort/version.

B. Bài tập tổng hợp – FUNews API Upgrade / Lab 03 Preparation

1. Tạo Spring Boot project có Spring Web, Data JPA, H2.
2. Tạo News entity tối thiểu: id, title, publishDate, active.
3. Tạo repository phù hợp; giải thích vì sao chọn JpaRepository.
4. Tạo service cho Page và Slice.
5. Tạo GET `/api/news` hỗ trợ page/size/sort.
6. Tạo GET `/api/news/slice` cho active news.
7. Tạo v1/v2 bằng ít nhất một versioning strategy.
8. Giới hạn size tối đa và whitelist sort field.
9. Test ít nhất 12 case bằng Postman.
10. Cập nhật README/API documentation với contract rõ ràng.

C. Acceptance checklist

Kiểm tra	Đạt khi
Versioning	v1/v2 trả contract khác có chủ đích
Paging	page 0/1 cho content khác nhau
Sorting	asc/desc quan sát được
Page metadata	totalElements/totalPages hợp lý
Slice	không phụ thuộc total count; hasNext đúng
Guard	size/sort invalid được xử lý
Layering	Controller → Service → Repository
Evidence	Postman + README + Git checkpoints

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
API Versioning	Quản lý nhiều contract/version của API	/api/v1/news
Backward compatibility	Client cũ vẫn dùng được contract mới/server mới	Giữ v1 trong giai đoạn migrate
Path versioning	Version nằm trong URL path	/v2/news
Query versioning	Version nằm trong query	?version=2
Custom header versioning	Version nằm trong header riêng	X-API-Version: 2
Media type versioning	Version nằm trong Accept/representation	application/vnd.funews.v2+json
Paging/Pagination	Chia collection thành các trang	page=0&size=10
Pageable	Thông tin page/size/sort	PageRequest.of(0,10)

PageRequest	Implementation của Pageable	PageRequest.of(...)
Sort	Tiêu chí sắp xếp	publishDate DESC
Page	Content + total metadata	totalPages=4
Slice	Content + next/previous metadata	hasNext=true
totalElements	Tổng số records	35
totalPages	Tổng số trang	4
hasNext	Còn dữ liệu phía sau?	true
PagingAndSortingRepository	Repository fragment cho paging/sorting	findAll(Pageable)
JpaRepository	Repository JPA giàu chức năng	NewsRepository extends JpaRepository
Offset pagination	Phân trang theo offset/page	page=100
Cursor/keyset pagination	Đi tiếp từ key/cursor	afterId=123
Stable sort	Sort nhất quán để paging không nhảy item	publishDate,id
Page size limit	Giới hạn số item/request	max 100
Spring Data web support	Resolve Pageable/Sort từ request params	page,size,sort
H2	DB in-memory cho demo	jdbc:h2:mem:slot15db
Lab 03	Lab REST Spring Boot bắt đầu ở Slot 15	Versioning/paging/sorting

VIII. RESEARCH BASE VÀ AI BASE LEARNING

A. 10 prompt nghiên cứu chuyên sâu

1. Phân tích ưu/nhược điểm của path, query, custom header và media type API versioning.
2. Giải thích repository hierarchy hiện tại của Spring Data: Repository, CrudRepository, PagingAndSortingRepository, JpaRepository.
3. Vì sao Page thường cần count query còn Slice không cần total count?
4. Phân tích cách PageableHandlerMethodArgumentResolver biến page/size/sort thành Pageable.
5. Phân tích stable sorting và lỗi duplicate/missing item khi dữ liệu thay đổi giữa các page request.
6. So sánh offset pagination, keyset pagination và cursor pagination.
7. Giải thích cách database index ảnh hưởng ORDER BY + pagination.
8. Phân tích JSON serialization của Page và lý do Spring Data khuyến nghị stable DTO/PagedModel.
9. Giải thích multi-column sorting với Spring Data Sort.Order.
10. Phân tích first-class API versioning trong Spring Framework mới và khác cách mapping conditions ở Spring Boot 3.

B. 10 prompt thực tiễn production

1. Khi nào một breaking change đủ lớn để tạo v2?
2. Làm migration/deprecation v1 → v2 thế nào để mobile client không bị ngắt?
3. Nên đặt page size default/max bao nhiêu và dựa vào chỉ số nào?
4. Làm sao whitelist sort fields mà vẫn dễ mở rộng?

5. Khi nào Page count query trở thành bottleneck?
6. Infinite scroll mobile nên dùng Slice hay cursor pagination?
7. Cách cache paginated API và invalidation khi data thay đổi?
8. API pagination có nên trả links next/prev hay chỉ metadata?
9. Làm sao test page boundary, invalid size, invalid sort và concurrent inserts?
10. Cách document paging/sorting/versioning bằng OpenAPI sao cho frontend dùng được ngay?

C. 10 prompt cập nhật công nghệ/xu hướng

1. So sánh Page, Slice, Window/Scroll APIs trong Spring Data hiện hành.
2. Spring Framework 7 API versioning hỗ trợ những resolver nào?
3. Keyset pagination trong Spring Data/JPA hiện được hỗ trợ ra sao?
4. GraphQL cursor pagination khác REST Page/Slice thế nào?
5. Elasticsearch search_after khác offset pagination thế nào?
6. Pagination trong MongoDB với skip/limit gặp vấn đề gì ở offset lớn?
7. Reactive pagination với WebFlux có cần Page không?
8. OpenAPI hiện mô tả pagination conventions như thế nào?
9. Các API lớn hiện dùng deprecation/sunset headers thế nào?
10. AI có thể sinh test cases cho pagination/versioning nhưng cần verification gì?

D. AI Verification Worksheet

Prompt/AI claim	Tôi kiểm tra ở đâu?	Experiment/code	Kết luận
Page luôn tốt hơn Slice	Spring docs + SQL logs	So sánh query	
Page index bắt đầu 1	Spring Data docs	GET page=0	
PagingAndSortingRepository extends JpaRepository	Javadoc hierarchy	Open IDE/Javadoc	
Header versioning = Accept header	HTTP/Spring mapping docs	Postman 2 variants	

AI rule: AI may generate; student must verify. Mọi prompt research phải kết thúc bằng ít nhất một bằng chứng: official docs, code chạy, Postman trace hoặc SQL log.

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Knowledge check

1. Breaking change là gì?
2. 4 chiến lược versioning và ví dụ mỗi loại?
3. Page index mặc định bao nhiêu?
4. Pageable gồm thông tin nào?
5. Sort có thể có nhiều Order không?
6. Page khác Slice ở total metadata thế nào?

7. Vì sao Slice có thể nhanh hơn?
8. PagingAndSortingRepository hiện nằm ở đâu trong hierarchy?
9. Vì sao cần max page size?
10. Khi nào dùng Page, khi nào dùng Slice?

2. Skill check

Tôi có thể...	Đạt	Chưa đạt	Cần làm lại
Tạo v1/v2 endpoint			
Nhận page/size/sort			
Tạo PageRequest			
Dùng JpaRepository paging			
Trả Page			
Trả Slice từ derived query			
Whitelist sort field			
Giới hạn page size			
Test bằng Postman			
Giải thích Page vs Slice			

3. Human Verification Gate

1. Mở code và chỉ ra nơi version được resolve.
2. Giải thích vì sao endpoint v2 khác v1 về contract.
3. Dự đoán response page=0,size=5 trước khi chạy.
4. Giải thích `PageRequest.of(0,10,Sort...)``.
5. Chỉ ra method repository nào trả Page.
6. Chỉ ra query nào trả Slice.
7. Giải thích vì sao Page có totalElements.
8. Sửa max size từ 100 xuống 50 và chứng minh.
9. Thêm sort field hợp lệ mới.
10. Tạo request sort field không hợp lệ và giải thích behavior.
11. Giải thích tại sao page vượt range thường trả empty page thay vì 404.
12. Nêu một case nên dùng Slice.
13. Nêu một case bắt buộc cần Page.
14. Nếu client v1 còn dùng, bạn retire v1 thế nào?
15. Nếu data lớn hơn 10 triệu rows, bạn sẽ nghiên cứu kỹ thuật pagination nào tiếp theo?

4. Exit Ticket

- Một khái niệm bạn hiểu rõ nhất hôm nay.

- Một lỗi versioning/paging bạn có thể tự debug.
- Một quyết định Page vs Slice và lý do.
- Một câu hỏi muốn mang sang Slot 16 testing.

5. Evidence Package

Mã	Bảng chứng
E1	Ảnh/response v1 và v2
E2	GET page=0
E3	GET page=1
E4	Sort asc/desc
E5	Page metadata
E6	Slice metadata
E7	Invalid sort/size experiment
E8	Repository/service/controller trace
E9	README/API contract
E10	AI verification log

PHỤ LỤC A – QUICK REFERENCE

Mục	Cú pháp/ghi nhớ
Page request	PageRequest.of(page, size, sort)
Default web params	page, size, sort
Sort	Sort.by(Direction.DESC, "publishDate")
Page	content + totalElements + totalPages
Slice	content + hasNext; không total count
Repository	JpaRepository cho demo JPA
Path version	/api/v1/news
Query version	?version=1
Header version	X-API-Version: 1
Media type	Accept: application/vnd.funews.v1+json

PHỤ LỤC B – POSTMAN VERIFICATION MATRIX

ID	Request/Test	Kết quả mong đợi
T01	GET /api/v1/news	200, contract v1
T02	GET /api/v2/news	200, contract v2
T03	GET page=0&size=5	5 items hoặc ít hơn

T04	GET page=1&size=5	content khác T03
T05	sort=title,asc	title tăng dần
T06	sort=title,desc	title giảm dần
T07	sort=publishDate,desc	mới nhất trước
T08	slice page=0	hasNext quan sát được
T09	page vượt range	empty content, no crash
T10	size quá lớn	clamp/reject theo contract
T11	sort field sai	400 hoặc handling đã định nghĩa
T12	query version sai	mapping/error phù hợp
T13	header version 1	contract v1
T14	Accept media v2	contract v2
T15	Swagger/OpenAPI nếu bật	page/size/sort được document

PHỤ LỤC C – KẾ HOẠCH 135 PHÚT

Thời lượng	Hoạt động
0–15	Retrieval Slot 14 + tình huống 10.000 News
15–35	Versioning strategies + mapping demo
35–55	Paging need + Pageable/Sort
55–75	Page vs Slice + repository correction
75–105	Demo H2/JPA paging/sorting + Postman
105–120	Break-It Lab + debug
120–130	Lab 03 kick-off + evidence
130–135	Exit Ticket

PHỤ LỤC D – CHECKPOINT GIT / PROJECT

1. V0 – Spring Boot project chạy.
2. V1 – Version endpoints hoạt động.
3. V2 – H2 + repository chạy.
4. V3 – Paging endpoint hoạt động.
5. V4 – Sorting + guard hoạt động.
6. V5 – Slice endpoint hoạt động.
7. V6 – Postman/README/evidence hoàn chỉnh.

Bridge sang Slot 16: Slot 15 đã xác định behavior cần đúng. Slot 16 sẽ chuyển các behavior này thành automated tests/MockMvc: version mappings, page boundaries, sort behavior và error cases.